



Universidad Nacional Experimental de Guayana.
Vicerrectorado Académico.
Coordinación General de Pregrado.
Proyecto de Carrera: Ingeniería en Informática.
Unidad Curricular: Ingeniería de Software II.

Fundamentos de la Gestión de Proyectos y Productos de Software.

Team Pixel & Code

“Estrategia, desarrollo y valor real”

Profesor:

Ing. Félix Márquez

Integrantes:

C.I.: V-28.530.717. Bravo Yvanna
C.I.: V-28.726.605. Guilarte Keibel
C.I.: V-19.505.397. Rivas Miguel
C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, 2 de Septiembre del 2026.

Ejercicio 1: Traducción de Necesidades a Requerimientos y Actores

La ingeniería de requisitos constituye el proceso mediante el cual se descubren, analizan, especifican, validan y gestionan las capacidades, restricciones y atributos de calidad que debe poseer un sistema de software. Según el SWEBOK V4 (IEEE Computer Society, 2014), esta disciplina delimita el vocabulario y los procesos propios de la profesión, integrando la elicitación, el análisis, la especificación y la validación de requisitos como actividades centrales del ciclo de vida del software. En el mismo sentido, el estándar internacional ISO/IEC/IEEE 29148 define las características que debe cumplir todo requerimiento bien redactado: ser unívoco, verificable, factible, necesario y trazable, y ofrece estructuras normalizadas para documentar la Especificación de Requisitos del Sistema (SyRS) y la Especificación de Requisitos de Software (SRS).

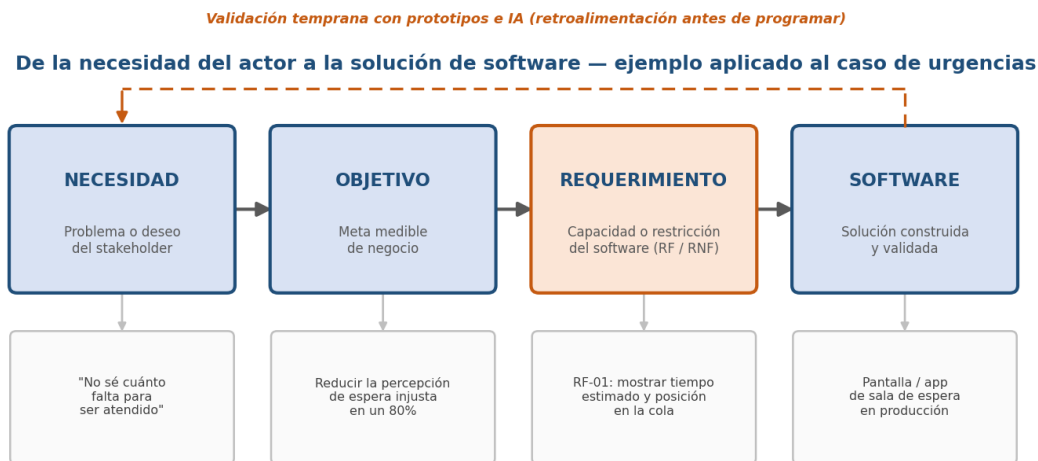


Figura 2. Trayecto de una necesidad del stakeholder hasta convertirse en una solución de software validada.

Dentro de este marco conviene distinguir tres nociones: las necesidades, que son los problemas o deseos subyacentes de los interesados (stakeholders); los objetivos, que son las metas medibles del negocio derivadas de esas necesidades; y los requerimientos, que son la traducción técnica de una necesidad en una capacidad concreta que el software debe ofrecer (funcional) o una condición que debe cumplir (no funcional). Los actores no se limitan a los usuarios finales: incluyen patrocinadores, desarrolladores, entes reguladores y, en general, a la sociedad cuando existe un impacto ético o social del sistema; por ello, el rol del Project Manager (PM) implica identificar tanto a los interesados favorables al proyecto como a aquellos con interés en que este no se concrete.

La importancia de gestionar correctamente esta etapa se sustenta en el concepto de costo del cambio: la curva de Boehm demuestra que corregir un defecto de requisitos durante la fase de mantenimiento puede costar hasta cien veces más que corregirlo durante la fase de elicitación

(Boehm, 1981). Un sistema con una arquitectura impecable pero que no resuelve la necesidad real del usuario constituye, en la práctica, un proyecto fracasado (Wiegiers & Beatty, 2013).

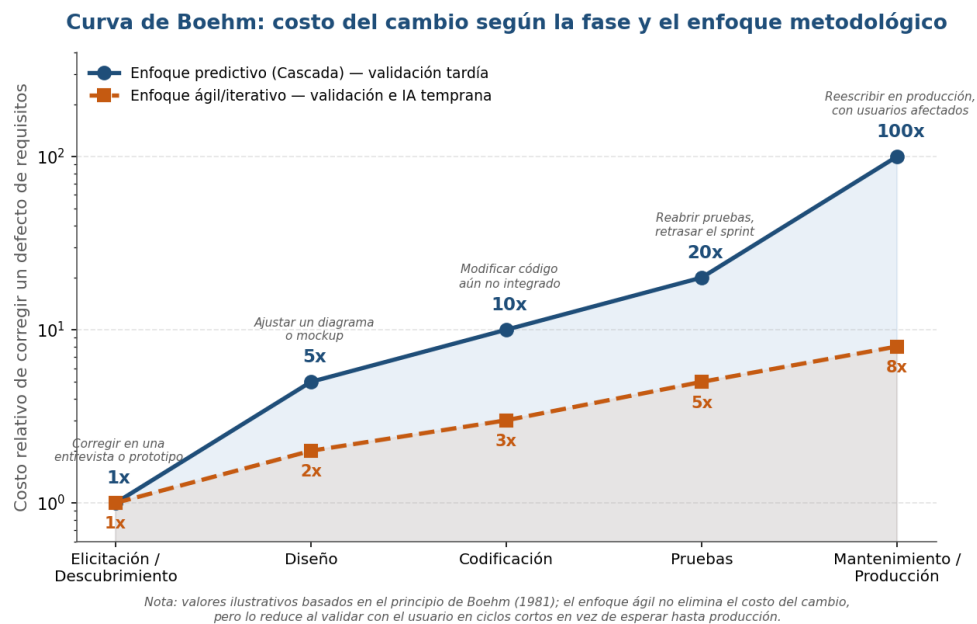


Figura 1. Costo relativo de corregir un defecto de requisitos según la fase del proyecto (Boehm, 1981).

En cuanto a las técnicas de elicitación, las clásicas entrevistas, cuestionarios, observación de campo y talleres colaborativos se complementan hoy con el prototipado rápido (por ejemplo, con Figma o Balsamiq) y con Historias de Usuario, que capturan el valor desde la perspectiva de quien usará el sistema (Wiegiers & Beatty, 2013). De forma más reciente, la Inteligencia Artificial Generativa se emplea para analizar grandes volúmenes de datos de uso real y sugerir requerimientos no funcionales o patrones de comportamiento que la elicitación tradicional podría no detectar, así como para simular conversaciones de validación antes de invertir en desarrollo.

Resolución del Ejercicio 1

Un hospital público necesita reducir el tiempo de espera de los pacientes en el área de urgencias. El problema no es únicamente clínico: involucra a personas con necesidades distintas y muchas veces contrapuestas (el paciente quiere certeza, el médico quiere orden clínico, la gerencia quiere eficiencia medible, TI quiere estabilidad y seguridad, y la sociedad quiere equidad). Antes de escribir una sola línea de código conviene aplicar el principio de "Interesados" del PMBOK 7 y la disciplina de la ingeniería de requisitos (ISO/IEC/IEEE 29148): identificar quién es cada actor, qué necesidad real tiene (no lo que "cree que quiere", para evitar el error clásico de falta de participación del usuario) y qué se le puede prometer sin caer en scope creep.

Se identifican a continuación seis actores (stakeholders): dos usuarios directos del proceso de urgencias (paciente y personal de triaje), dos roles internos del hospital (gerencia y TI), y dos actores indirectos pero legítimos (el familiar/acompañante y la sociedad representada por el ente regulador), reconociendo como señala el material del curso que un actor no es solo quien está interesado en que el proyecto se concrete, sino cualquiera con interés en su resultado.

Mapa de actores: poder vs. Interés

Antes de definir requerimientos conviene priorizar a quién escuchar primero. La siguiente matriz clasifica a los seis actores según su poder/influencia sobre el proyecto y su nivel de interés en el resultado, lo que orienta la estrategia de comunicación de cada uno (gestionar de cerca, mantener satisfecho, mantener informado o monitorear).

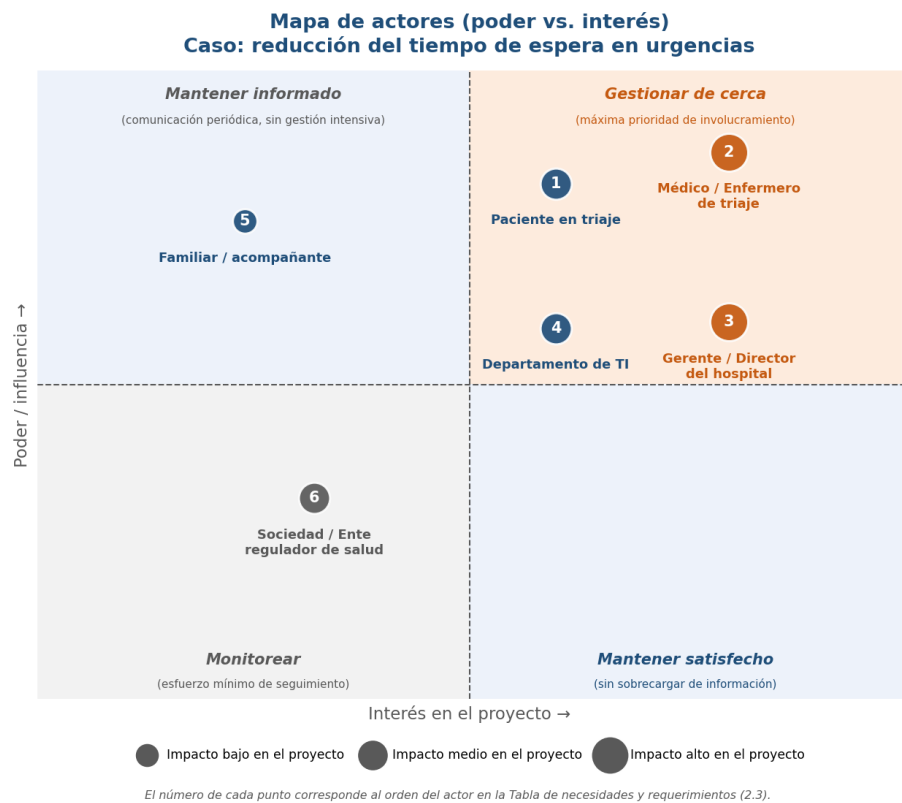


Figura 3. Mapa de actores del caso de urgencias, clasificados por poder e interés.

Actores, necesidades y requerimientos

La siguiente tabla traduce cada necesidad identificada en un requerimiento específico, indicando si es funcional (define una capacidad que el sistema debe hacer) o no funcional (define una condición de calidad, seguridad o desempeño que el sistema debe cumplir).

Actor (Stakeholder)	Necesidad (problema identificado)	Requerimiento derivado
Paciente en triaje	No sabe cuánto tiempo falta para ser atendido y percibe la espera como injusta o insegura.	RF-01 (Funcional): El sistema debe mostrar en una pantalla o app el tiempo estimado de espera y la posición del paciente en la cola de triaje, actualizado en tiempo real.
Médico/Enfermero de triaje	Debe clasificar rápidamente la gravedad de cada paciente sin perder trazabilidad ni cometer errores por sobrecarga.	RF-02 (Funcional): El sistema debe permitir registrar el nivel de urgencia (ej. escala Manchester) de cada paciente en menos de 2 minutos y priorizar automáticamente la cola según ese nivel.
Gerente / Director del hospital	Necesita indicadores objetivos para justificar decisiones de personal y presupuesto, y demostrar mejora del servicio.	RNF-01 (No funcional - Usabilidad/Reportaría): El sistema debe generar reportes de tiempo promedio de espera, tiempo de atención y volumen de pacientes por turno, exportables y accesibles desde un panel gerencial.
Departamento de TI del hospital	Debe garantizar que el sistema esté disponible 24/7 y que los datos de pacientes estén protegidos.	RNF-02 (No funcional - Disponibilidad/Seguridad): El sistema debe tener una disponibilidad mínima del 99.5% y cumplir con cifrado y control de acceso a los datos clínicos conforme a normativa de protección de datos de salud.
Familiar / acompañante del paciente	Se siente ansioso e ignorado porque no recibe información sobre el estado o el avance de la atención de su familiar.	RF-03 (Funcional): El sistema debe permitir consultar, mediante un código de seguimiento, el estado general del paciente (en espera, en atención, derivado) sin exponer datos clínicos sensibles.
Sociedad / Ente regulador de salud	Interés en que el sistema de urgencias sea equitativo, auditable y no genere discriminación en la	RNF-03 (No funcional - Cumplimiento/Auditoría): El sistema debe registrar un log auditable de cada

Actor (Stakeholder)	Necesidad (problema identificado)	Requerimiento derivado
	atención.	decisión de priorización, de modo que pueda verificarse que la atención se otorgó según criterio clínico y no por otro factor.

Especificación ampliada: historias de usuario y criterios de aceptación

Para que el requerimiento quede totalmente claro y sea verificable tal como exige ISO/IEC/IEEE 29148 cada uno se redacta también como historia de usuario, con su criterio de aceptación y su prioridad bajo el esquema MoSCoW (Must have, Should have, Could have, Won't have this time). Esto conecta cada línea de la tabla anterior con un producto concreto y verificable, no solo con una idea general.

Actor	Historia de usuario	Criterio de aceptación	Prioridad
Paciente en triaje	Como paciente en espera, quiero ver mi tiempo estimado de espera y mi posición en la cola, para sentir certeza y reducir mi ansiedad.	Dado que el paciente fue clasificado en triaje, cuando consulta la pantalla o app, entonces ve un tiempo estimado (± 10 min) y su posición, actualizados al menos cada 2 minutos.	Must
Médico/Enfermero de triaje	Como enfermero de triaje, quiero registrar el nivel de urgencia en menos de 2 minutos, para priorizar correctamente sin retrasar la atención de casos graves.	Dado un paciente evaluado, cuando el enfermero registra el nivel de la escala Manchester, entonces el sistema reordena automáticamente la cola en menos de 5 segundos.	Must
Gerente / Director del hospital	Como gerente del hospital, quiero un panel con tiempos promedio de espera y atención por turno, para tomar decisiones de personal basadas en datos.	Dado un rango de fechas seleccionado, cuando el gerente abre el panel, entonces visualiza y puede exportar (PDF/Excel) los indicadores de espera, atención y volumen de pacientes.	Should

Actor	Historia de usuario	Criterio de aceptación	Prioridad
Departamento de TI	Como responsable de TI, quiero que el sistema tenga alta disponibilidad y datos cifrados, para garantizar continuidad del servicio y cumplimiento normativo.	Dado el sistema en producción, cuando se mide en un mes calendario, entonces la disponibilidad es $\geq 99.5\%$ y los datos clínicos viajan y se almacenan cifrados.	Must
Familiar / acompañante	Como familiar de un paciente, quiero conocer el estado general de su atención con un código de seguimiento, para no sentirme ignorado sin acceder a datos clínicos privados.	Dado un código de seguimiento válido, cuando el familiar lo ingresa, entonces ve un estado general (en espera / en atención / derivado) sin detalles clínicos.	Could
Sociedad / Ente regulador	Como ente regulador, quiero que cada decisión de priorización quede registrada y sea auditable, para verificar que la atención se basó en criterio clínico y no en otro factor.	Dado cualquier cambio de prioridad en la cola, cuando ocurre, entonces se guarda en un log auditable con usuario, criterio clínico aplicado y marca de tiempo.	Should

Matriz de trazabilidad

La trazabilidad permite demostrar, ante una auditoría o un cambio de alcance, por qué existe cada requerimiento y a quién beneficia. Resume el recorrido completo: **actor** → **necesidad** → **objetivo medible** → **requerimiento** → **prioridad**.

Actor	Necesidad	Objetivo medible	Requerimiento (ID)	Prioridad
Paciente en triaje	Incertidumbre sobre el tiempo de espera.	Reducir la percepción de espera injusta en un 80%.	RF-01	Must
Médico/Enfermero	Clasificar rápido sin errores por	Clasificar cada paciente en ≤ 2 min	RF-02	Must

Actor	Necesidad	Objetivo medible	Requerimiento (ID)	Prioridad
	sobrecarga.	con 0 reprocesos.		
Gerente / Director	Necesita datos de objetivos de gestión.	Reducir el tiempo promedio de espera en un 25% en 6 meses.	RNF-01	Should
Departamento de TI	Garantizar continuidad y protección de datos.	Mantener disponibilidad $\geq$ 99.5% y 0 incidentes de fuga de datos.	RNF-02	Must
Familiar / acompañante	Sentirse informado sin acceder a datos clínicos.	Reducir consultas presenciales al mostrador en un 50%.	RF-03	Could
Sociedad / Ente regulador	Garantizar equidad y trazabilidad de la atención.	100% de las priorizaciones auditables ante requerimiento legal.	RNF-03	Should

Reflexión: el rol de la IA y el prototipado en la validación

Antes de programar, el prototipado rápido permite construir maquetas de baja o media fidelidad (por ejemplo, la pantalla de tiempo estimado de espera para el paciente, o el panel de indicadores del gerente) que pueden mostrarse a los propios actores para verificar si realmente comunican lo que necesitan, evitando construir software técnicamente correcto pero inútil para el usuario real. La Inteligencia Artificial Generativa aporta una capa adicional: puede analizar históricos de admisión en urgencias para sugerir umbrales realistas de tiempo de espera, detectar patrones no evidentes (por ejemplo, picos de demanda en ciertos horarios) y ayudar a redactar historias de usuario y criterios de aceptación más completos, reduciendo la ambigüedad de los requerimientos (Wiegers & Beatty, 2013). En ambos casos, el objetivo es validar el requerimiento con retroalimentación temprana de los actores, aplicando el principio de "Interesados" del PMBOK 7 involucrar eficazmente a los stakeholders para comprender sus necesidades antes de comprometer recursos de desarrollo, lo que reduce el riesgo de retrabajo y de deuda técnica descrito en la curva de Boehm (Figura 1).

Aplicado al caso concreto: un prototipo de la pantalla de "tiempo estimado de espera" mostrado al propio paciente y al personal de triaje permitiría descubrir, antes de programar, si el formato (minutos, posición en cola, o ambos) realmente reduce su ansiedad o si, por el contrario, genera más preguntas de las que resuelve. De igual forma, un modelo de IA entrenado con el histórico de admisiones del hospital podría anticipar, por ejemplo, que los lunes en la mañana el tiempo de espera real supera sistemáticamente al estimado, permitiendo ajustar el algoritmo de RF-01 antes de que se convierta en una fuente de desconfianza. Esta validación temprana es, en esencia, la aplicación práctica de la curva de Boehm: cuesta casi nada corregir un prototipo en papel o una simulación con datos históricos, y cuesta hasta cien veces más corregir esa misma suposición equivocada una vez el sistema ya está en producción y siendo usado por pacientes reales.

Ejercicio 2: Análisis de las 4 dimensiones y errores clásicos

Enunciado

Una startup decide desarrollar una app de delivery. El gerente técnico impone el uso de una arquitectura de microservicios en Kubernetes (tecnología) pero el equipo solo sabe programar en PHP monolítico (personas). No definieron un proceso de gestión de cambios (proceso) y el cliente pide agregar una función de pagos con criptomonedas a mitad del desarrollo sin ajustar el presupuesto (producto y alcance).

Se pide analizar los desbalances en las 4 dimensiones de Pressman, identificar al menos 3 errores clásicos presentes en el escenario y proponer una mitigación inmediata desde el rol de un gerente de proyectos moderno entendido como líder servicial.

Desbalances en las cuatro dimensiones

Pressman explica que el software funciona como un ecosistema de cuatro dimensiones que dependen entre sí, de modo que cuando una avanza sin las demás el proyecto entero se desbalancea. Eso es lo que ocurre en este caso, porque cada decisión parece defendible cuando se la mira por separado pero ninguna se coordinó con el resto y el efecto conjunto termina siendo mucho peor que la suma de sus partes.

- 1. Tecnología:** La arquitectura elegida no está mal en abstracto, ya que una app de delivery termina necesitando escalar por separado sus módulos de pedidos, geolocalización y pagos, y los microservicios responden bien a esa necesidad. El problema aparece cuando la decisión se toma sin mirar el resto del proyecto, porque Kubernetes exige orquestación de contenedores, diseño de servicios independientes y observabilidad distribuida, y nada de eso se evaluó antes de imponerlo. Ahí es donde falla el pensamiento sistémico que el PMBOK 7 plantea como uno de sus principios.

2. **Personas:** Este es el desbalance más grave, porque el equipo domina PHP monolítico y pasar de ahí a microservicios implica cambiar la forma de pensar el sistema completo. Hay que repensar cómo se guardan los datos, qué ocurre cuando falla un solo servicio y cómo se despliega todo eso, y el equipo simplemente no tiene esa experiencia. Cuando la tecnología se define sin medir la capacidad real de la gente se genera una deuda de aprendizaje que después se paga con retrabajo y con entregas mucho más lentas de lo previsto.
3. **Proceso:** No existe un procedimiento para evaluar solicitudes de cambio y aunque suene a trámite administrativo esa carencia es la causa directa del siguiente problema, ya que sin ese filtro la función de criptomonedas entra al desarrollo sin que nadie mida cuánto cuesta ni qué riesgo trae. El proceso es lo que debería regular a las otras tres dimensiones y al no existir cualquier decisión se propaga sin control.
4. **Producto.** El producto deja de responder a una idea clara de valor y pasa a ser la suma de decisiones sueltas, y la función de criptomonedas lo muestra bien porque no agrega solo una pantalla más sino que trae consigo requisitos de seguridad y de cumplimiento legal que debieron analizarse con el mismo rigor que la norma ISO/IEC/IEEE 29148 exige para cualquier requerimiento del sistema.

Errores clásicos identificados

Optimismo injustificado: Se asume que el equipo va a poder con Kubernetes sin que nadie lo verifique ni pregunte cuánto tiempo toma aprenderlo, y lo que viene después lo describe bien la ley de Hofstadter. El atraso no se nota al principio y aparece más adelante, cuando los plazos empiezan a caerse sin que exista un error de código puntual que explique por qué.

Scope creep (desvío de alcance): El cliente pide criptomonedas a mitad del desarrollo y la petición se acepta sin tocar presupuesto ni cronograma, aunque el problema real no está en que pida cambios porque eso es normal y hasta deseable en un entorno adaptativo. Lo que falla es que no hay ninguna forma de medir el impacto antes de decir que sí, y esa diferencia es justamente la que separa un cambio bien gestionado de un alcance que se desborda, que es a lo que solemos llamar “scope creep”.

Falta de participación en las decisiones: Ni el equipo opinó sobre la arquitectura ni al cliente se le explicó lo que su pedido implicaba, de manera que se terminó decidiendo sobre los actores en lugar de decidir con ellos. El PMBOK 7 llama a esto involucramiento de interesados y lo coloca entre sus principios porque su ausencia rompe proyectos que técnicamente eran viables.

Mitigación desde el rol de líder servicial

La salida no pasa por detener el desarrollo ni por imponer otra decisión desde arriba sino por devolverle coherencia a las cuatro dimensiones, y para eso hay tres acciones que pueden tomarse de inmediato.

La primera es sentarse con el equipo y definir en conjunto si los microservicios se sostienen desde el inicio o si conviene una migración por partes que respete la curva de aprendizaje real, porque una decisión construida con el equipo se sostiene mejor que una impuesta. La segunda es levantar un procedimiento simple de solicitud de cambios que obligue a estimar tiempo, costo y riesgo antes de aceptar cualquier funcionalidad nueva, con lo cual el proyecto recupera el filtro que hoy le falta. La tercera es volver a negociar con el cliente el alcance o el presupuesto para incorporar los pagos con criptomonedas, dejando claro que responder al cambio es un valor del Manifiesto Ágil pero no significa aceptar todo sin mover las condiciones del proyecto.

El líder servicial no resuelve este caso imponiendo criterio técnico sino quitando obstáculos, protegiendo al equipo de una presión que no corresponde con su capacidad real y devolviéndole al proyecto un mínimo de orden entre lo que se quiere construir, con qué se va a construir y quiénes lo van a construir.

Visión, estándar y tópico aplicado

Se trabajó con las 4 dimensiones de Pressman como marco de diagnóstico y con los errores clásicos documentados por Brooks y por el CHAOS Report del Standish Group, mientras que la propuesta de mitigación se apoya en los principios de pensamiento sistémico, interesados y administración del PMBOK 7, en la norma ISO/IEC/IEEE 29148 para el tratamiento de requisitos y en el liderazgo servicial de Greenleaf como rol del gerente de proyectos moderno.

Ejercicio 3: PMBOK 6 (basado en procesos) vs. PMBOK 7 (basado en principios) y su impacto en la gestión de requisitos de software

El enfoque del PMBOK 6: gestión basada en procesos

La sexta edición del PMBOK estructura la dirección de proyectos como un conjunto cerrado de 49 procesos, organizados en cinco grupos (inicio, planificación, ejecución, monitoreo y control, y cierre) y diez áreas de conocimiento, entre ellas la Gestión de los Requisitos dentro del área de Alcance. Cada proceso se describe mediante el esquema entradas–herramientas y técnicas–salidas, lo que otorga un carácter prescriptivo: el estándar indica qué actividad ejecutar, en qué momento del ciclo de vida y qué artefacto debe producirse. Bajo esta lógica, el éxito del proyecto se mide principalmente por el cumplimiento del alcance, el costo y el tiempo planificados, el llamado Triángulo de Hierro, asumiendo que la conformidad con el plan inicial garantiza la utilidad de la solución entregada.

El enfoque del PMBOK 7: gestión basada en principios

La séptima edición del PMBOK (PMI, 2021) representa una transformación paradigmática:

abandona la estructura de procesos y áreas de conocimiento para adoptar un marco de 12 principios rectores y 8 dominios de desempeño. Entre los principios destacan la administración diligente (Stewardship), el trabajo en equipo colaborativo, el involucramiento efectivo de los interesados, el enfoque continuo en el valor, el pensamiento sistémico, el liderazgo, la adaptación del enfoque (tailoring) al contexto, la calidad, la gestión de la complejidad, la optimización de respuestas ante el riesgo, la adaptabilidad y resiliencia, y la habilitación del cambio. En lugar de prescribir procedimientos rígidos, el PMBOK 7 concibe el proyecto como un subsistema dentro de un sistema mayor de entrega de valor organizacional, priorizando el logro de resultados (outcomes) por encima de la conformidad procesal, y es aplicable indistintamente a proyectos predictivos, híbridos o ágiles.

Cuadro comparativo

Criterio	PMBOK 6 (basado en procesos)	PMBOK 7 (basado en principios)
Estructura	49 procesos organizados en 5 grupos y 10 áreas de conocimiento	12 principios rectores y 8 dominios de desempeño
Enfoque central	Cumplimiento del plan: alcance, costo y tiempo (Triángulo de Hierro)	Entrega continua de valor (outcomes) por encima de la conformidad procesal
Naturaleza	Prescriptiva: indica qué hacer y en qué orden (input-herramienta-salida)	Adaptativa: indica qué se debe lograr, dejando el cómo a criterio del equipo
Aplicabilidad	Orientado a proyectos predictivos (cascada)	Aplicable a proyectos predictivos, híbridos y ágiles mediante tailoring
Gestión de requisitos	Documento de requisitos como línea base formal, sujeta a control de cambios	Requisitos como hipótesis de valor, revisadas iterativamente con los interesados

Impacto del cambio de enfoque en la gestión de requisitos de software

Este tránsito de un modelo prescriptivo a uno principista transforma sustancialmente la manera en que se gestionan los requisitos de software. Bajo el PMBOK 6, la gestión de requisitos se concibe como un proceso formal y secuencial: se elicitán, documentan y aprueban en una línea base (baseline) que luego se somete a control de cambios mediante procedimientos administrativos, típicamente dentro de un ciclo de vida predictivo o en cascada. El requisito, una vez congelado, se convierte en un contrato que debe cumplirse tal como fue especificado, y

cualquier modificación implica reabrir un proceso formal de gestión de cambios.

Bajo el PMBOK 7, en cambio, los requisitos se tratan como hipótesis de valor que deben validarse continuamente con los interesados a lo largo del proyecto, en línea con los principios de involucramiento de los interesados, entrega de valor y adaptación. La gestión de requisitos deja de ser un evento único de aprobación para convertirse en un ciclo permanente de descubrimiento, retroalimentación y ajuste, coherente con la ingeniería de requisitos definida en la norma ISO/IEC/IEEE 29148, que exige que todo requerimiento sea unívoco, verificable, factible, necesario y trazable, pero que (a diferencia del enfoque de línea base rígida) puede refinarse en cada iteración sin que ello se interprete como una desviación del plan, sino como una respuesta legítima al principio de adaptabilidad y resiliencia.

Ejemplo 1: Cambio de un requisito a mitad de proyecto

En un proyecto gestionado bajo PMBOK 6, si el cliente solicita agregar una nueva funcionalidad (por ejemplo, pagos con criptomonedas en una app de delivery) después de haber aprobado la línea base de alcance, esa solicitud debe pasar obligatoriamente por el proceso formal "Realizar el Control Integrado de Cambios", con evaluación de impacto en costo y cronograma antes de su aceptación o rechazo. En un proyecto gestionado bajo PMBOK 7, el mismo cambio se evalúa a la luz del principio de "Enfoque en el Valor" y de "Adaptabilidad y Resiliencia": el equipo prioriza el nuevo requisito dentro del backlog en función del valor de negocio que aporta, y lo incorpora en la siguiente iteración sin necesidad de reabrir un expediente formal de control de cambios, siempre que exista transparencia con los interesados.

Ejemplo 2: Validación de requisitos con los interesados.

Bajo PMBOK 6, la validación de requisitos ocurre en un punto concreto del ciclo de vida (típicamente durante la planificación), mediante la firma de un documento de especificación de requisitos de software (SRS) que sirve de referencia formal para las siguientes fases. Bajo PMBOK 7, en aplicación del principio de "Interesados" y "Equipo", la validación es continua: por ejemplo, un Product Owner presenta incrementos funcionales al final de cada sprint para que los usuarios finales confirmen o ajusten el requerimiento (mediante prototipos en Figma o revisiones de sprint), de modo que el requisito se refina de forma empírica en lugar de basarse en una única aprobación documental temprana.

Ejercicio 4: El riesgo de construir lo que nadie usa

En la ingeniería de software moderna, el mayor riesgo no es que el código no compile, sino que el software se construya perfectamente, pero nadie lo use. Discuta esta afirmación relacionándola con la importancia de la ingeniería de requisitos, la identificación de actores y la crisis del software.

Por Miguel Rivas,

Según el texto recibido de la unidad I, considero las siguientes relaciones para cada uno de los puntos a tratar:

- 1) **Respecto a la Crisis del software**, es importante considerar que desde los años 60 se hasta principios del 2000 se *“buscaba la solución perfecta, pero para el problema equivocado”*, esto sucedía porque el enfoque estaba puesto en entregar software a tiempo y buscando la minimización de errores sintácticos, además; de presupuestos ajustados.

En la actualidad, con la evolución de los compiladores y la introducción de frameworks, el fracaso ocurre por la inutilidad del producto entregado.

- 2) **En cuanto a la ingeniería de requisitos**, considero que se falla en la elicitation y especificación de requisitos, estaremos desarrollando *“sistemas técnicamente impecables pero desalineados con las necesidades de negocio”*.

- 3) **Si fallamos en la identificación y mapeo de actores**, se pueden *“generar barreras de usabilidad, resistencia al cambio y rechazo”*. Para entender un poco mejor este punto y el anterior considero importante analizar la siguiente imagen:



Esta imagen el ciclo de desarrollo de software actual, como podemos notar es un ciclo iterativo, pues según los cambios requeridos se realizan ajustes en el producto entregado, y se retroalimenta el ciclo. Entonces para concluir, si cometemos errores en los puntos 2) y 3) estamos fallando en la entrada de dicha retroalimentación, obteniendo al final un producto que no satisface las necesidades del usuario final.